

Database Team - IT Support Ticket Resolution Records

Confidential - Internal Use Only | Database Operations Team | 2024

[INCIDENT] DB-001 - Production database query running extremely slow

Date: 2024-01-05

Priority: Critical

Category: Performance

Type: Incident

Description:

Main product listing query on PostgreSQL taking over 45 seconds to return results. Affecting all users on the platform. Query involves joining orders, products, and inventory tables. Customer complaints coming in through support channel.

Resolution / Action Taken:

Investigation revealed missing composite index on orders table for (customer_id, created_at). Created index using: `CREATE INDEX CONCURRENTLY idx_orders_customer_date ON orders(customer_id, created_at)`. Query time dropped from 45s to 0.3s. Also ran `ANALYZE` on affected tables to update query planner statistics. Notified engineering team to review other high-frequency queries.

[INCIDENT] DB-002 - Primary database server unresponsive, full outage

Date: 2024-01-08

Priority: Critical

Category: Availability

Type: Incident

Description:

Primary PostgreSQL server completely unresponsive at 3:14AM. All application services down. Monitoring shows server reachable via ping but PostgreSQL port 5432 not responding. On-call engineer paged.

Resolution / Action Taken:

SSH into server revealed PostgreSQL process had crashed due to out-of-memory kill. Server had 16GB RAM, `shared_buffers` was set too high at 12GB leaving insufficient memory for connections. Restarted PostgreSQL. Adjusted `shared_buffers` to 4GB and `work_mem` to 64MB. Promoted read replica to primary while main server stabilized. Total downtime 23 minutes. Implemented memory alerts and auto-restart systemd config.

[INCIDENT] DB-003 - Accidental deletion of records from customers table

Date: 2024-01-12

Priority: Critical

Category: Data Loss

Type: Incident

Description:

Developer ran `DELETE` query without `WHERE` clause on production customers table during maintenance. Approximately 2,300 records deleted. Immediate rollback attempted but transaction already committed. Business impact severe.

Resolution / Action Taken:

Restored deleted records from automated daily backup taken at 2AM. Used point-in-time recovery to restore to state 10 minutes before deletion. Total data loss window was 6 hours. 47 new customers re-onboarded manually by support team. Implemented query review policy requiring peer approval for `DELETE` and `UPDATE` on production. Added `pg_audit` extension for all DML query logging.

[INCIDENT] DB-004 - Read replica lagging behind primary by over 2 hours

Date: 2024-01-15

Priority: High

Category: Replication

Type: Incident

Description:

Monitoring alerts showing replica lag of 7,200 seconds on read replica used for reporting. Reports showing stale data. Finance team unable to generate accurate daily reports. Primary database load is normal.

Resolution / Action Taken:

Large batch import job on primary generating very high WAL volume. Replica unable to keep up. Paused batch import, replica caught up in 25 minutes. Restarted import at reduced speed with rate limiting. Long term: scheduled batch imports during off-peak hours 2AM-4AM. Increased `max_wal_senders` and `wal_keep_size` parameters on primary.

Database Team - IT Support Ticket Resolution Records

Confidential - Internal Use Only | Database Operations Team | 2024

[INCIDENT] DB-005 - Application unable to connect to database after deployment

Date: 2024-01-18

Priority: High

Category: Connectivity

Type: Incident

Description:

After deploying new backend version, application throwing connection refused errors. Database server running fine. Other services connecting without issues. New deployment went live 20 minutes ago.

Resolution / Action Taken:

New deployment had incorrect DATABASE_URL in environment variables. Connection string pointing to staging database instead of production. Updated environment variable to correct production host. Connection pool size was also set to 2 which was too low - increased to 20. Added database connection health check to deployment pipeline.

[INCIDENT] DB-006 - Deadlock errors appearing frequently in application logs

Date: 2024-01-22

Priority: High

Category: Performance

Type: Incident

Description:

Application logs showing deadlock detected errors every 5 minutes. Affecting order processing. Two transactions locking each other causing one to rollback each time. Order failures increasing.

Resolution / Action Taken:

Analyzed deadlock logs using pg_locks and pg_stat_activity. Found two stored procedures updating inventory and orders tables in opposite order. Proc A locks inventory then orders, proc B locks orders then inventory. Fixed by standardizing lock acquisition order in both procedures to always lock inventory first then orders. Deadlocks reduced to zero immediately.

[INCIDENT] DB-007 - Suspicious brute force login attempts on database server

Date: 2024-01-28

Priority: High

Category: Security

Type: Incident

Description:

Security monitoring flagged 847 failed login attempts from external IP 185.220.x.x over 2 hours. Attempting to brute force postgres superuser account. Security team escalated to database team.

Resolution / Action Taken:

Immediately blocked IP range at firewall level. Confirmed superuser uses strong password and certificate auth - no successful logins. Audited all DB user accounts, removed 3 stale accounts from departed employees. Restricted port 5432 to VPN IP range only. Enabled fail2ban for PostgreSQL. Rotated all database passwords as precaution. Filed security incident report.

[INCIDENT] DB-008 - Automated backup job failing silently for 5 days

Date: 2024-02-02

Priority: Medium

Category: Backup

Type: Incident

Description:

Weekly backup verification check found automated nightly backups had been failing for 5 days with no alerts triggered. Last successful backup is 5 days old. Compliance team notified as this violates RPO policy.

Resolution / Action Taken:

Backup script failing because S3 bucket permissions changed during security audit, removing write access for backup IAM role. Re-added s3:PutObject permission. Ran manual backup immediately. Fixed script to properly propagate error codes instead of always returning exit 0. Added backup success verification: if no backup file in S3 within 25 hours, page on-call engineer.

Database Team - IT Support Ticket Resolution Records

Confidential - Internal Use Only | Database Operations Team | 2024

[INCIDENT] DB-009 - Schema migration failed halfway, database in inconsistent state

Date: 2024-02-05

Priority: High

Category: Migration

Type: Incident

Description:

Alembic migration to add new columns to transactions table failed midway due to timeout. Table now has partial schema. Application failing to start. Engineering team blocked on release.

Resolution / Action Taken:

Inspected current schema state directly. Manually rolled back partial migration by dropping newly added columns. Fixed migration script to use smaller batch sizes and explicit timeouts. Re-ran migration during low traffic window 1AM with statement_timeout set to 300s. Migration completed in 8 minutes. Going forward all migrations require staging test, maintenance window, and rollback script.

[INCIDENT] DB-010 - Connection pool exhausted during peak traffic flash sale

Date: 2024-02-14

Priority: Medium

Category: Connectivity

Type: Incident

Description:

During flash sale event application reporting connection pool exhausted errors. Database has max_connections 100 but application trying to open 200+ connections. Requests timing out, revenue impact ongoing.

Resolution / Action Taken:

Immediate: restarted application servers to clear idle connections. Long term: implemented PgBouncer connection pooler in transaction mode. PgBouncer maintains 20 persistent DB connections serving 500 application connections efficiently. Reduced app-level pool size from 50 to 10 per instance. Max concurrent DB connections now stable at 25 even during peak load.

[SERVICE REQUEST] DB-011 - New data analyst requires read access to reporting database

Date: 2024-01-10

Priority: Medium

Category: Access

Type: Service Request

Description:

New team member Sarah Johnson joined data analytics team on January 10th. Requires SELECT access to all tables in reporting schema. Manager approval received. Access needed by end of day.

Resolution / Action Taken:

Created database role analyst_read with SELECT permissions on all tables in reporting schema: GRANT SELECT ON ALL TABLES IN SCHEMA reporting TO analyst_read. Assigned role to user sarah.johnson. Granted USAGE on schema. Set default privileges so future tables automatically accessible. Sent credentials securely via password manager. Documented in access log.

[SERVICE REQUEST] DB-012 - Revoke database access for departed employee

Date: 2024-01-14

Priority: Low

Category: Access

Type: Service Request

Description:

HR notified that developer Marcus Webb left the company on January 13th. Request to immediately revoke all database access including application user and any personal developer accounts. Compliance requirement.

Resolution / Action Taken:

Identified all database accounts associated with Marcus Webb: mwebb_dev, mwebb_readonly, and shared dev account access. Dropped personal accounts: DROP USER mwebb_dev; DROP USER mwebb_readonly. Rotated password on shared dev account since user had access. Confirmed no active sessions at time of revocation. Updated access log and notified HR and security team of completion.

Database Team - IT Support Ticket Resolution Records

Confidential - Internal Use Only | Database Operations Team | 2024

[SERVICE REQUEST] DB-013 - Request temporary elevated access for quarterly audit

Date: 2024-01-20 Priority: Medium Category: Access Type: Service Request

Description:

External auditors from KPMG require read access to transaction and financial tables for Q4 2023 audit. Access required for 2 weeks starting January 22nd. Legal and compliance team have approved. Must be time-limited.

Resolution / Action Taken:

Created temporary audit role: `CREATE ROLE kpmg_audit_q4 WITH LOGIN VALID UNTIL '2024-02-05'`. Granted `SELECT` on transactions, payments, invoices, and audit_log tables only. Created dedicated read-only database user `audit_kpmg_2024` assigned to this role. Provided credentials to compliance team for handoff. Set calendar reminder to revoke access on February 5th. Access revoked on schedule.

[SERVICE REQUEST] DB-014 - Developer needs access to staging database for new feature development

Date: 2024-01-25 Priority: Low Category: Access Type: Service Request

Description:

Developer Priya Sharma starting work on new payments feature. Needs full read/write access to staging database payments schema. Production access not required. Team lead has approved the request.

Resolution / Action Taken:

Granted full privileges on staging database payments schema to `priya.sharma` user account: `GRANT ALL PRIVILEGES ON SCHEMA payments TO priya.sharma IN DATABASE staging`. No production access granted. Reminded developer that staging data is refreshed weekly from sanitized production copy. Access documented in team access registry.

[CHANGE REQUEST] DB-015 - Request to increase production database server disk size from 1TB to 4TB

Date: 2024-01-16 Priority: High Category: Infrastructure Type: Change Request

Description:

Production database disk at 78% capacity. At current growth rate of 15GB per week, will reach 90% in approximately 8 weeks. Requesting disk upgrade before hitting critical threshold. Approved by infrastructure manager.

Resolution / Action Taken:

Submitted AWS support request to increase EBS volume from 1TB gp2 to 4TB gp3. AWS expanded volume without downtime. Extended filesystem using `resize2fs` command. Verified new capacity reflected correctly. Also took opportunity to upgrade from gp2 to gp3 which improved baseline IOPS from 3,000 to 12,000 at lower cost. New alerts set at 70% and 85% thresholds. Total process completed in 2 hours with zero downtime.

[CHANGE REQUEST] DB-016 - Upgrade PostgreSQL version from 13 to 16 on production

Date: 2024-01-19 Priority: Medium Category: Infrastructure Type: Change Request

Description:

PostgreSQL 13 approaching end of life in November 2025. Requesting upgrade to PostgreSQL 16 to stay current and benefit from performance improvements. Security team flagged this as required. Staging upgrade already completed successfully.

Resolution / Action Taken:

Scheduled maintenance window Saturday 2AM-6AM. Used `pg_upgrade` for in-place upgrade to minimize downtime. Pre-upgrade: full backup taken, all extensions compatibility verified, application connection strings noted. Upgrade completed in 35 minutes. Post-upgrade: ran `ANALYZE` on all databases, verified all extensions working, ran application smoke tests. Application restarted and

Database Team - IT Support Ticket Resolution Records

Confidential - Internal Use Only | Database Operations Team | 2024

verified healthy. Total planned downtime 2 hours, actual 45 minutes.

[CHANGE REQUEST] DB-017 - Request to increase max_connections from 100 to 300 on production

Date: 2024-01-26	Priority: Medium	Category: Configuration	Type: Change Request
------------------	------------------	-------------------------	----------------------

Description:

Engineering team reports application occasionally hitting connection limits during peak hours. Requesting increase of max_connections parameter from 100 to 300 to handle growth. Product team expects 3x traffic increase after upcoming launch.

Resolution / Action Taken:

Analyzed current connection usage patterns first. Peak usage was 87 connections. Simply increasing max_connections would require proportionally more shared memory. Instead recommended PgBouncer connection pooler as better solution. Deployed PgBouncer, configured pool size of 30 persistent connections. Application can now handle 500 concurrent app connections mapped to 30 DB connections. More efficient than raising max_connections. Engineering team satisfied with solution.

[CHANGE REQUEST] DB-018 - Increase statement timeout from 30 seconds to 120 seconds for reporting queries

Date: 2024-02-01	Priority: Low	Category: Configuration	Type: Change Request
------------------	---------------	-------------------------	----------------------

Description:

Data team reports that complex quarterly reports are being killed by the 30 second statement timeout. Reports involve aggregating 2 years of transaction data. Requesting timeout increase specifically for reporting database user.

Resolution / Action Taken:

Instead of globally increasing statement timeout which would affect all queries, applied timeout change at the role level: ALTER ROLE reporting_user SET statement_timeout = '120s'. This applies only when reporting_user connects, not affecting other users. Also worked with data team to optimize the slowest reports using materialized views and pre-aggregated tables, reducing the longest report from 95s to 18s. Only 2 reports still need the extended timeout.

[CHANGE REQUEST] DB-019 - Add second read replica for analytics workload separation

Date: 2024-02-08	Priority: Medium	Category: Infrastructure	Type: Change Request
------------------	------------------	--------------------------	----------------------

Description:

Current single read replica serving both application read queries and analytics team queries. Analytics workload causing replica lag during business hours affecting application performance. Request to provision dedicated analytics replica.

Resolution / Action Taken:

Provisioned new read replica on larger instance type (r6g.2xlarge vs existing r6g.large) optimized for analytics workloads with higher RAM. Configured replica with different postgresql.conf tuned for analytical queries: higher work_mem, larger shared_buffers. Updated analytics team connection strings to point to new replica. Original replica now exclusively serves application read traffic. Replica lag on app replica dropped from average 45s to under 1s.

[CHANGE REQUEST] DB-020 - Increase RDS instance size from db.r6g.large to db.r6g.4xlarge

Date: 2024-02-12	Priority: High	Category: Infrastructure	Type: Change Request
------------------	----------------	--------------------------	----------------------

Description:

Database team forecasting capacity constraints ahead of Black Friday event. Current instance showing 75% CPU during normal load. Requesting upgrade to larger instance to handle 10x expected traffic spike. Business critical event in 3 weeks.

Database Team - IT Support Ticket Resolution Records

Confidential - Internal Use Only | Database Operations Team | 2024

Resolution / Action Taken:

Analyzed CPU, memory, and IOPS metrics over past 30 days. CPU bottleneck confirmed. Scheduled instance resize during low traffic Sunday 3AM. Used RDS modify-db-instance command. Downtime was 8 minutes during instance class change. New instance has 16 vCPUs vs 2, and 128GB RAM vs 16GB. Post-upgrade CPU baseline dropped to 12%. Stress tested with simulated Black Friday load - peaked at 45% CPU. Capacity confirmed adequate. Scheduled downsize after event to optimize costs.

[PROBLEM] DB-021 - Recurring slow query on transactions table every morning 9AM

Date: 2024-01-30 Priority: Medium Category: Performance Type: Problem

Description:

Every morning between 9AM and 9:15AM database performance degrades. Same query on transactions table doing full sequential scan. Issue resolves itself but causes poor user experience daily. Pattern observed for 3 weeks.

Resolution / Action Taken:

Root cause: autovacuum running statistics collection on transactions table every morning triggered by overnight batch job writing millions of rows. During autovacuum, query planner using stale statistics and choosing sequential scan over index. Fixed by: 1) Scheduled manual ANALYZE to run at 8:50AM before business hours. 2) Tuned autovacuum to run more frequently in smaller increments rather than large morning run. 3) Created index on frequently filtered columns. Morning degradation eliminated.

[PROBLEM] DB-022 - Database transaction logs growing unbounded, disk filling weekly

Date: 2024-02-06 Priority: High Category: Storage Type: Problem

Description:

WAL (Write Ahead Log) files consuming increasing disk space. Operations team manually cleaning logs weekly. Not sustainable. Root cause unknown. Logs should be archived or cleaned automatically.

Resolution / Action Taken:

Investigation found WAL archiving was enabled but archive_command was failing silently because S3 archive bucket was in wrong region. Failed archive commands cause PostgreSQL to retain all WAL files indefinitely. Fixed archive_command to point to correct S3 bucket. All backlogged WAL files archived and old ones cleaned up, recovering 180GB. Implemented archive success monitoring. WAL retention now stable at approximately 2GB.

[PROBLEM] DB-023 - Full table scan happening on large transactions table repeatedly

Date: 2024-02-10 Priority: Medium Category: Performance Type: Problem

Description:

Query explain plan showing sequential scan on transactions table with 50 million rows. Query filters by status and created_at but not using index despite index existing. Problem intermittent - sometimes uses index, sometimes does not.

Resolution / Action Taken:

Found that table statistics were significantly outdated causing planner to underestimate row counts and choose sequential scan. Ran ANALYZE to update statistics. Created partial index specifically for active transactions: CREATE INDEX idx_transactions_active ON transactions(created_at) WHERE status IN (pending, processing). Also rewrote query to avoid SELECT star. Query time reduced from 3.5 minutes to 0.8 seconds consistently.

[PROBLEM] DB-024 - Tables showing significant bloat after large delete operations

Date: 2024-02-16 Priority: Low Category: Maintenance Type: Problem

Database Team - IT Support Ticket Resolution Records

Confidential - Internal Use Only | Database Operations Team | 2024

Description:

After data cleanup job deleting 8 million old records, table sizes on disk unchanged. pg_stat_user_tables showing high dead tuple counts. Queries slightly slower. Storage not being reclaimed despite deletes completing successfully.

Resolution / Action Taken:

PostgreSQL DELETE does not immediately reclaim disk space - dead tuples cleaned by VACUUM. Ran VACUUM ANALYZE on affected tables. For most bloated tables ran VACUUM FULL during off-peak hours to fully compact storage. Adjusted autovacuum settings: reduced autovacuum_vacuum_scale_factor from 0.2 to 0.05 for high-churn tables so autovacuum triggers more frequently. Space reclaimed, query performance restored.

[ENHANCEMENT] DB-025 - Request for query performance monitoring dashboard

Date: 2024-01-17	Priority: Low	Category: Monitoring	Type: Enhancement
------------------	---------------	----------------------	-------------------

Description:

Team lacks visibility into slow queries. Issues discovered reactively only when users complain. Request to set up proactive monitoring so team can identify and fix slow queries before they impact users.

Resolution / Action Taken:

Enabled pg_stat_statements extension to track query execution statistics. Created Grafana dashboard showing: top 20 slowest queries by average execution time, queries with highest total time, queries with most calls per minute, cache hit ratios, and connection count trends. Set up weekly slow query report emailed to team every Monday. Configured log_min_duration_statement to 1000ms. Within first week identified and fixed 3 slow queries proactively.

[ENHANCEMENT] DB-026 - Implement point-in-time recovery capability for production database

Date: 2024-01-23	Priority: Medium	Category: Backup	Type: Enhancement
------------------	------------------	------------------	-------------------

Description:

Current backup strategy is daily snapshots only. Recovery point objective is 24 hours which is too long. Business requires RPO of 1 hour maximum. Request to implement continuous backup with point-in-time recovery.

Resolution / Action Taken:

Configured continuous WAL archiving to S3 using archive_command. Enabled pg_basebackup for daily base backups. Set wal_level to replica and archive_mode to on. Tested point-in-time recovery by restoring to specific timestamps - successfully recovered to within 5 minutes of any point. Documented recovery procedure with step by step runbook. Recovery tested quarterly now. RPO reduced from 24 hours to under 5 minutes.

[ENHANCEMENT] DB-027 - Request to implement connection pooling for application database connections

Date: 2024-01-29	Priority: Medium	Category: Performance	Type: Enhancement
------------------	------------------	-----------------------	-------------------

Description:

Development team reports application opening and closing database connections very frequently causing overhead. Each new connection takes 50-100ms to establish. Request to implement connection pooling to reuse existing connections.

Resolution / Action Taken:

Deployed PgBouncer as connection pooler in transaction pooling mode on same server as application. Configured pool of 25 persistent connections to database. Application now connects to PgBouncer port 6432 instead of database port 5432. Connection establishment time dropped from 50-100ms to under 1ms since connections are reused. Application throughput improved by 18%. PgBouncer admin interface set up for monitoring pool stats.

Database Team - IT Support Ticket Resolution Records

Confidential - Internal Use Only | Database Operations Team | 2024

[ENHANCEMENT] DB-028 - Set up automated database failover for high availability

Date: 2024-02-03 Priority: High Category: Infrastructure Type: Enhancement

Description:

Current setup has no automatic failover. If primary database goes down, manual intervention required which takes 20-30 minutes. Business requires RTO of 5 minutes. Request to implement automated failover.

Resolution / Action Taken:

Deployed Patroni cluster manager with etcd for distributed consensus. Configured streaming replication with synchronous_commit for zero data loss. Patroni monitors primary health every 10 seconds and triggers automatic failover within 30 seconds if primary unreachable. Updated application to use HAProxy virtual IP that automatically routes to current primary. Tested failover by killing primary - application reconnected in 28 seconds automatically. RTO reduced from 30 minutes to under 1 minute.

[ENHANCEMENT] DB-029 - Request to automate regular database maintenance tasks

Date: 2024-02-07 Priority: Low Category: Maintenance Type: Enhancement

Description:

Database team spending 4 hours every weekend manually running VACUUM, ANALYZE, REINDEX on key tables. Request to automate this maintenance to free up team time and ensure consistency.

Resolution / Action Taken:

Created automated maintenance scripts using pg_cron extension. Scheduled: ANALYZE on all tables every day at 1AM, VACUUM on high-churn tables every 6 hours, REINDEX CONCURRENTLY on bloated indexes every Sunday 2AM, pg_stat_reset monthly. All jobs log results to maintenance_log table. Grafana alert if any maintenance job fails or takes longer than expected. Weekend manual maintenance eliminated. Team saves 4 hours every week.

[ENHANCEMENT] DB-030 - Implement database activity auditing and compliance logging

Date: 2024-02-11 Priority: Medium Category: Security Type: Enhancement

Description:

Compliance team requires full audit trail of all data access and modifications on production database for SOC2 certification. Need to log who accessed what data and when. Currently no auditing in place.

Resolution / Action Taken:

Installed and configured pgaudit extension. Configured to log all DDL statements, DML on sensitive tables (customers, payments, personal_data), all authentication events, and privilege grants. Logs shipped to centralized SIEM via log aggregation pipeline. Created compliance dashboard showing access patterns and anomalies. Log retention set to 1 year per compliance requirement. SOC2 audit passed with auditing evidence provided.

[INFORMATION REQUEST] DB-031 - What is the current database size and growth rate?

Date: 2024-01-11 Priority: Low Category: General Type: Information Request

Description:

Product manager requesting current database size, growth rate over past 6 months, and forecast for next 12 months to inform infrastructure budget planning for next fiscal year.

Resolution / Action Taken:

Pulled database size metrics from monitoring: Current total size 847GB across all databases. Main application DB is 612GB, reporting DB 180GB, audit logs 55GB. Growth rate over past 6 months averaging 18GB per week. At current rate will reach 1.5TB in

Database Team - IT Support Ticket Resolution Records

Confidential - Internal Use Only | Database Operations Team | 2024

approximately 37 weeks. Recommended budget for: disk upgrade to 4TB in Q2, consider database archiving strategy for records older than 3 years which would reduce size by estimated 30%. Provided full report to product manager.

[INFORMATION REQUEST] DB-032 - Provide database schema documentation for new development team

Date: 2024-01-21 Priority: Low Category: General Type: Information Request

Description:

New development team onboarding and needs complete database schema documentation including table descriptions, relationships, column definitions, and indexes. No documentation currently exists.

Resolution / Action Taken:

Generated schema documentation using pg_dump and custom queries against information_schema. Created comprehensive documentation covering: all 47 tables with descriptions and row counts, primary and foreign key relationships as ER diagram, all indexes with rationale, stored procedures and their purposes, naming conventions used. Published to internal Confluence wiki. Set up automated weekly schema change diff so documentation stays current. Onboarding team confirmed documentation sufficient.

[INFORMATION REQUEST] DB-033 - Report on database performance metrics for past quarter

Date: 2024-02-04 Priority: Low Category: Performance Type: Information Request

Description:

Engineering manager requires quarterly database performance report for Q4 2023 to present to leadership. Should include average query times, slow query trends, uptime, incident summary, and capacity utilization.

Resolution / Action Taken:

Compiled Q4 2023 database performance report: Average query response time 45ms (target 100ms - exceeding target). Uptime 99.94% (2 incidents totaling 31 minutes downtime). Slowest query improved from 12s to 0.3s after indexing work. Peak connection utilization 87%. Storage growth 18GB/week. Incidents: 1 OOM crash, 1 accidental deletion. Improvements made: 4 new indexes, PgBouncer deployed, backup verification added. Report formatted as PDF and delivered to manager.

[INFORMATION REQUEST] DB-034 - Audit of all database users and their permission levels

Date: 2024-02-15 Priority: Medium Category: Security Type: Information Request

Description:

Security team conducting annual access review. Requires complete list of all database users, their roles, what databases and schemas they can access, and when their access was last used. Part of annual security audit.

Resolution / Action Taken:

Ran comprehensive access audit using pg_roles, pg_auth_members, information_schema.role_table_grants. Identified 23 active users across all databases. Found 4 accounts with no login activity in over 90 days - confirmed with HR that 3 belonged to departed employees, 1 was legacy service account no longer needed. Disabled all 4 inactive accounts. Created spreadsheet with all users, roles, permissions, last login, and business justification. Delivered to security team. Recommended quarterly access reviews going forward.

[INCIDENT] DB-035 - Database disk usage at 95 percent, approaching critical threshold

Date: 2024-02-18 Priority: High Category: Storage Type: Incident

Description:

Database Team - IT Support Ticket Resolution Records

Confidential - Internal Use Only | Database Operations Team | 2024

Alert triggered showing primary database server disk at 95% usage. Database 800GB on 1TB disk. Risk of database going read-only if disk fills completely. Immediate action required.

Resolution / Action Taken:

Immediate: identified and deleted 3 months of stale temporary tables left by old ETL jobs, freed 120GB. Ran VACUUM FULL on bloated tables recovering additional 45GB. Total immediate recovery 165GB, disk back to 63% usage. Long term: enabled pg_partman for automatic table partitioning. Set up automated archival of records older than 2 years to S3 cold storage. Upgraded disk to 4TB. Alerts now set at 70% and 85% thresholds.

[SERVICE REQUEST] DB-036 - Request to restore specific deleted records from backup

Date: 2024-02-20	Priority: Low	Category: Data	Type: Service Request
------------------	---------------	----------------	-----------------------

Description:

Business analyst accidentally deleted 150 rows from the campaigns table while testing a cleanup script. Records contain important campaign configuration data. Requests restoration of these specific records without affecting rest of database.

Resolution / Action Taken:

Performed selective restore using point-in-time recovery on a separate temporary database instance restored to 1 hour before deletion. Extracted the 150 deleted records using SELECT on temporary instance. Inserted recovered records back into production campaigns table with original IDs preserved. Verified record count and data integrity. Temporary instance terminated. Total recovery time 35 minutes. User counseled on using transactions and testing in staging first.

[CHANGE REQUEST] DB-037 - Enable SSL/TLS encryption for all database connections

Date: 2024-02-22	Priority: Medium	Category: Configuration	Type: Change Request
------------------	------------------	-------------------------	----------------------

Description:

Security audit identified that database connections not enforcing SSL encryption. Sensitive customer data potentially transmitted in plain text. Compliance team requires all database connections encrypted. Change requested by CISO.

Resolution / Action Taken:

Generated SSL certificates using AWS Certificate Manager. Configured PostgreSQL ssl = on in postgresql.conf. Set ssl_cert_file and ssl_key_file. Added hostssl entries to pg_hba.conf replacing host entries to require SSL. Updated all application connection strings to include sslmode=require. Updated database URL in all services. Tested all connections verify SSL using pg_stat_ssl view. All 23 active connections now showing SSL encrypted. Compliance team signed off.

[ENHANCEMENT] DB-038 - Implement materialized views for slow reporting queries

Date: 2024-02-25	Priority: Low	Category: Performance	Type: Enhancement
------------------	---------------	-----------------------	-------------------

Description:

Finance team running 6 reports that each take 10-20 minutes to generate. Reports aggregate large amounts of historical transaction data. Users complain reports too slow. Request to investigate optimization.

Resolution / Action Taken:

Analyzed all 6 slow reports. Found 4 of them query the same large aggregation of 3 years of transactions joined with products and categories. Created materialized view pre-computing this aggregation: CREATE MATERIALIZED VIEW mv_transaction_summary AS... Scheduled REFRESH MATERIALIZED VIEW CONCURRENTLY every night at 1AM. Reports using this view now return in under 2 seconds vs previous 15-20 minutes. Remaining 2 reports optimized with partial indexes. All 6 reports now complete in under 5 seconds.

Database Team - IT Support Ticket Resolution Records

Confidential - Internal Use Only | Database Operations Team | 2024

[SERVICE REQUEST] DB-039 - Export production data sample for machine learning team

Date: 2024-02-27 Priority: Medium Category: Data Type: Service Request

Description:
ML team requires sample of 500,000 anonymized transaction records for training fraud detection model. Data must be anonymized to remove PII before export. Data governance team has approved the request.

Resolution / Action Taken:
Created anonymization script replacing customer names with fake names using faker library, hashing email addresses and phone numbers, generalizing precise location to city level, removing exact dates replacing with week numbers. Exported 500,000 randomly sampled records meeting criteria. Ran data quality checks verifying no PII remains. Generated export as CSV and uploaded to ML team S3 bucket with appropriate access permissions. Data governance team reviewed and approved anonymized dataset.

[CHANGE REQUEST] DB-040 - Migrate database from on-premise server to AWS RDS

Date: 2024-03-01 Priority: High Category: Infrastructure Type: Change Request

Description:
Company infrastructure strategy moving to cloud. Request to migrate production PostgreSQL database from on-premise hardware to AWS RDS PostgreSQL. On-premise server lease expiring in 60 days. Migration must be completed with minimal downtime.

Resolution / Action Taken:
Migration plan: 1) Provisioned RDS PostgreSQL 16 instance in same region as application servers. 2) Set up AWS DMS continuous replication from on-premise to RDS. 3) Ran parallel for 2 weeks verifying data consistency. 4) Scheduled 30 minute maintenance window for final cutover. 5) Stopped writes to on-premise, let DMS sync final transactions, updated application connection strings, restarted application. Cutover took 18 minutes. Zero data loss confirmed. On-premise server decommissioned. Performance improved 40% due to better hardware and network proximity to application.